МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хиймэл оюуны үндэс

Лекц № 5

This e-learning content was developed by the support of KOICA

МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хичээл 5
Хайлт ба оновчлол

Компьютерын ухааны салбар

This e-learning content was developed by the support of KOICA

Энэ удаагийн лекцээр

- Хайлт гэж юу вэ?
- Хайлтын алгоритмын хэллэг ;
- Хайлтын стратегийн шалгуур үзүүлэлтүүд;
- Модоор хайгч алгоритмууд;
- Мод өгөгдлийн бүтэц;
- Хайлтын алгоритмыг дүрслэх ;
- Хиймэл оюун ухааны хайлтын алгоритмын төрөл;
- Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм;
- Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм;
- Дүгнэлт.

Ж.Алимаа. ШУТИС, МХТС, Компьютерийн ухааны салбар, дэд профессор (PhDДоктор). (e-mail: alimaaj@must.edu.mn)



Суралцахуйн үр дүн

Хиймэл оюун ухааны асуудлыг шийдвэрлэх аргачлалуудыг судалж тэдгээрийн практик хэрэглээг жишээгээр тайлбарласан агуулгатай танилцах болно. Үүнд:

- Хайлтын тодорхойлолт, түүнд хэрэглэгддэг нэр томъёо;
- Хайлтын алгоритмууд тэдгээрийн ангилал, онцлог, хэрэглэх аргачлалын талаарх ойлголт;
- Хайлтын алгоритмуудын давуу ба сул талууд, хэрэглэх боломжуудын талаарх мэдлэг эзэмших болно.

Хайлт гэж юу вэ?

- Хиймэл оюунд асуудлыг шийдвэрлэх аргачлалуудын нэг нь хайлт юм.
- Төлөвүүдийн олонлогийг ашиглан асуудлыг шийддэг. Асуудлыг шийдвэрлэх гэдэг нь хүссэн зорилгодоо хүрэх эсвэл тухайн нөхцөл байдалд шийдэл олох арга гэж нэрлэгддэг.
- Компьютерын шинжлэх ухаанд асуудал шийдвэрлэх гэдэг нь хиймэл оюун ухааны аргуудыг хэлдэг бөгөөд үүнд үр дүнтэй алгоритм бүрдүүлэх, эвристик , үндсэн шалтгааны шинжилгээ хийх зэрэг нь зүйтэй бөгөөд үүнд шаардлагатай шийдлийг олох болно.
- Хайлтын алгоритмууд нь тэдгээрийн өгсөн шийдлийн чанар дээр үндэслэн бүрэн бүтэн байдал, оновчтой байдал, хугацааны хүндрэл, орон зайн хүндрэлийн шинж чанарыг агуулдаг.

Хайлт гэж юу вэ?

Амжилт

- Амжилттай тоглож болох программууд нь хайлтад суурилсан байдаг.
- Бусад төрлийн хиймэл оюуны асуудлууд хайлтаар амжилттай шийдвэрлэгдэж байдаг.

Практик

- Ихэнх асуудлууд энгийн алгоритмаар шийдвэрлэгддэггүй.
- Эдгээр асуудлуудыг шийдвэрлэхэд бэрхшээлтэй байдаг тул тэдгээрийг шийдэх хамгийн хялбар арга нь хайлт байдаг.
- Хайлт нь барагцаалахад их ач холбогдолтой.
- Оновчлолын асуудал нь дотоод хайлт болно.

Хайлт гэж юу вэ?

Ихэнх тусгайлсан алгоритмууд нь нэмэлт мэдлэгүүдийг ашиглан хялбар өөрчлөгдөх боломжгүй байдаг. Гар аргаар буюу механик хайлт нь нэмэлт мэдлэгийг ашиглахыг шаарддаг.

- Ухаалаг байхын чухал нь төлөвлөлт байдаг. Төлөвлөлтийг хайлтаар шийддэг;
- Хайлт нь тооцооллын арга юм;
- Гэхдээ, хайлтаар шийдвэрлэгддэггүй олон асуудлууд байдаг; Хайлтаар загварчлагдаагүй асуудлууд бол шийдэгдэж чадахгүй;
- Асуудлыг хэрхэн зөв шийдвэрлэхийг нь загварчилснаар хайлт хэрэгждэг.

Хайлтын алгоритмуудын хэллэг

1. Хайлт нь өгөгдсөн хайлтын талбар дахь хайлтын асуудлыг алхам алхмаар шийдвэрлэдэг. Хайлтын асуудал нь гурван үндсэн хүчин зүйлтэй:
    - **Хайлтын зам**: системд байж болох олон шийдлүүдийн багцыг илэрхийлнэ.
    - **Эхлэл төлөв**: агент хайж эхлэх зангилаа.
    - **Зорилтын тест**: энэ нь одоогийн төлөв байдлыг ажиглаж, зорилгын төлөвт хүрсэн эсэхийг шалгах функц.
2. **Хайлтын мод**: Хайлтын асуудлын модны дүрслэлийг хайлтын мод гэж нэрлэдэг. Хайлтын модны үндэс нь анхны төлөвт харгалзах үндэс зангилаа юм.
3. **Үйлдэл**: Энэ нь боломжит бүх үйлдлийн тодорхойлолтыг агентад өгдөг.

Хайлтын алгоритмуудын хэллэг

4. **Шилжилтийн загвар**: Үйлдэл бүр юу хийдэг талаарх тодорхойлолтыг дүрсэлнэ. Нэг үйлдлийг хийж зангилааг өргөтгөх шилжилтийн загвар юм.

5. **Замын өртөг**: Энэ нь зам бүрд тоон зардлыг хуваарилдаг функц юм.

6. **Шийдэл**: Энэ бол эхлэх цэгээс зорилгын цэг рүү чиглэсэн үйлдлийн дараалал юм.

7. **Оновчтой шийдэл**: Хэрэв шийдэл нь бүх шийдлүүдийн дунд хамгийн бага өртөгтэй бол оновчтой шийдэл болно.

Хиймэл оюун ухааны хайлтын стратеги

Хайлтын стратегиудыг дараах үндсэн шалгуур үзүүлэлтүүдээр үнэлж болно:

- **Completeness /Төгсгөлөг байдал/:** Энэ стратеги нь шийдлийг баталгаатай олох уу?
- **Time complexity /Хугацааны хүндрэл/:** Шийд олохдоо хир удаж байна?
- **Space complexity /Санах ойн хүндрэл/:** Хайлтыг гүйцэтгэхдээ хир зэрэг санах ой хэрэглэж байна?
- **Optimality /Оновчтой эсэх/:** Асуудалд хэд хэдэн шийдэл байвал хамгийн чанартай шийдийг олж чадаж байна уу?

Хугацаа болон санах ойн хүндрэл нь дараах хүчин зүйлсээр хэмжигдэнэ:

- **b/branching factor/:** Хайлтын модны максимум фактор. Нэг зангилаанаас гарах максимум зангилааны тоог уг модны фактор гэе.
- **d/depth/:** Шийдийн орших гүн.
- **m/maximum/:** Хайлтын орон зайн/state space/ максимум гүн

Хиймэл оюун ухааны хайлтын стратегийн шалгуур үзүүлэлтүүд

Шалгуур	BFS	UCS	DFS	DLS	IDS	BDS	Greedy	A*	A* graph	Graph search
Хугацааны хүндрэл	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$	b^d	b^d	b^d	b^d
Санах ойн хүндрэл	b^d	b^d	$b*m$	$b*l$	$b*d$	$b^{d/2}$	b^d	b^d	b^d	b^d
Оновчтой байдал	Yes	Yes	No	No	Yes	Yes	No	Yes	Yes	Yes
Бүрэн бүтэн байдал	Yes	Yes	No	Yes Хэрэв, (l>=d)	Yes	Yes	No	Yes	Yes	Yes

Модоор хайгч алгоритм

Санаа: Тухайн төлөвөөс операторуудаа ашиглан шинээр төлөвүүд гарган өргөтгөх процесс юм. Энэ процессыг модыг өргөтгөх буюу **expanding** гэж хэлдэг.

Эндээс хайлтын мод (**search tree**) гэсэн ойлголт гарч ирдэг. Хүн ч гэсэн асуудлыг шийдэх дээ ийм хайлтын мод ашиглан сэтгэдэг.

Тухайн төлөвөөс салбарласан төлөвүүдээс аль төлөвийг эхэлж өргөтгөхийг сонгох процессыг хайлтын стратеги (**strategy**) гэдэг.

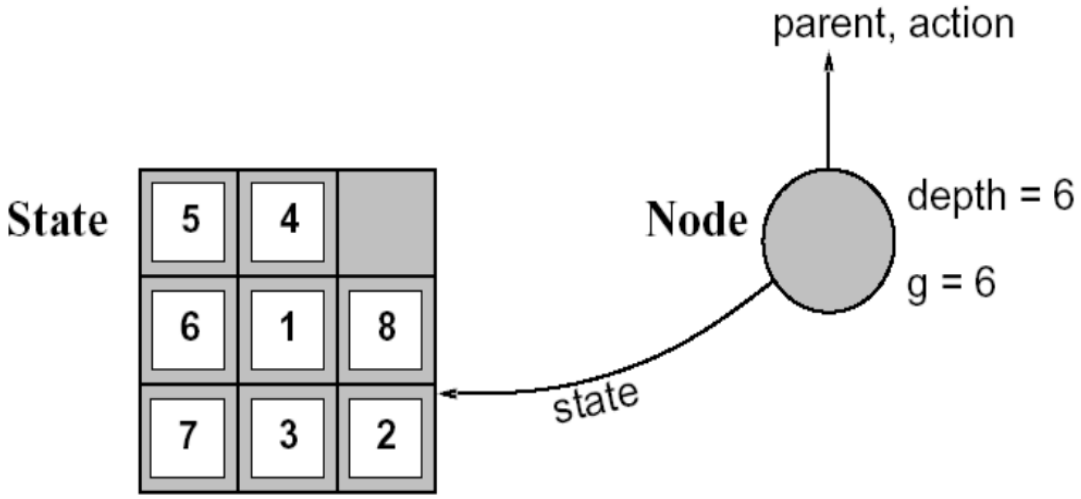
Жишээ: модоор хайх алгоритмын функц

```
def tree_search(problem,fringe):
   #Зорилгоо олохын тулд асуудлын залгамж халааг хайж ол.
   # fringe аргумент нь хоосон дараалал байх ёстой.
      fringe.append(Node(problem.initial))
      while fringe:
          node=fringe.pop()
          if problem.goal_test(node.state):
              return node

fringe.extend(node.expand(problem))
       return None
```

Мод өгөгдлийн бүтэц

Мод нь **зангилаанууд ба холбоосоос** тогтоно.
Төлөв нь **физик ертөнцийн шинж чанаруудын** цуглуулга.
Зангилаа нь **эцэг, хүү, гүн, замын зардал g(x)** зэргийг агуулсан өгөгдлийн бүтэц юм.
Төлөвт бол эцэг, хүү, гүн... байдаггүй.



Зураг 8. Мод өгөгдлийн бүтцийг Puzzle дээр үзүүлэв.
Эх сурвалж:Д.Золзаяа, Хиймэл оюун ба машин сургалт сурах бичиг, х-67

Expand функц нь **Successor Fn-**ийг ашиглан төрөл бүрийн төлөвүүдэд харгалзах шинэ зангилаануудыг үүсгэдэг.
Өргөтгөгдөхөөр хүлээгдэж байгаа зангилаануудын олонлогийг **fringe** эсвэл **frontier** гэж нэрлэдэг.
Хайлтын стратеги нь эдгээрээс дараачийн өргөтгөгдөх зангилааг сонгодог.
Энэхүү **fringe** зангилаануудын олонлогийг дараалал байдлаар (**Queue**) дүрсэлдэг. Үүний тулд дараах функцүүд хэрэгтэй:

- **Make-Queue(elements):** Өгөгдсөн элементүүтэй дараалал үүсгэнэ.
- **Empty?(Queue):** Дараалал хоосон үед үнэн утга буцаана.
- **Remove-Front(Queue):** Дарааллын эхэнд байх элементийг салган авч буцаана.
- **Queuing-Fn(Elements,Queue):** Элементүүдийг дараалалд нэмнэ.

Хайлтын алгоритмуудыг дүрслэх

Граф хайлт

Өмнө дурьдсанчлан хайлт гэдэг нь тодорхой зорилгод нийцсэн шийдлийг олохын тулд боломжит шийдлүүдийг судлах үйл явц юм. Граф хайлт нь зам эсвэл шийдлийг олохын тулд графын зангилаа ба ирмэгийг судлах үйл явцыг багтаана.

Граф гэж юу вэ?

Зангилаанууд (Оройнууд): Систем доторх байгууллагууд эсвэл элементүүдийг төлөөлөх (жишээлбэл, газрын зураг дээрх хотууд, нийгмийн сүлжээн дэх хүмүүс, вэб хуудас).

Ирмэгүүд (Холболт): Зангилаа хоорондын харилцаа, холболтыг (жишээ нь, хот хоорондын зам, нөхөрлөл, гипер холбоос) төлөөлдөг.

Чиглүүлсэн/Чиглүүлээгүй: Ирмэгүүд нь чиглэсэн (нэг талын харилцаа) эсвэл чиглүүлээгүй (хоёр талын харилцаа) байж болно.

Жинтэй/Жингүй: Ирмэгүүд нь харилцааны өртөг, зай, хүч чадлыг илэрхийлэх жинтэй байж болно.

Яагаад граф ашиглах вэ?

Граф нь харилцан уялдаатай объект, харилцааг хамарсан өргөн хүрээний асуудлыг загварчлах маш энгийн бөгөөд олон талын арга юм. Эдгээр нь системийн холболт, урсгал болон бусад шинж чанарыг шинжлэх боломж олгодог бүтцийг гаргаж өгдөг.

График хайлтын гол алгоритмууд

Бидний авч үзэх хайлтын алгоритмууд (**BFS, DFS, UCS, A***) нь график хайлтанд түгээмэл хэрэглэгддэг бөгөөд графын бүтэцтэй ажиллахад зориулагдсан байдаг.

Хиймэл оюун ухааны хайлтын алгоритмын төрөл

Граф хайлтын үндсэн санаа

Граф хайлт гэдэг нь графын холболт дээр үндэслэсэн дарааллаар графын орой ба ирмэг дээр очиж үзэх алгоритмын схем юм. Граф нь холбоосоор холбогдсон боловч давталт хийх боломжтой ба зангилаа нь олон эцэг эх байж болно.

Хоёр төрлийн граф буюу чиглэлтэй эсвэл чиглэлгүй холбоостой граф гэж байдаг. Граф хайлтын асуудлуудыг модны хайлтын асуудал болгон хувиргаж болно.

Граф нь дараах 2 бүрэлдэхүүн хэсгээс бүрдэх өгөгдлийн төрөл юм. Үүнд:

- Оройнууд буюу зангилаануудын олонлог;
- Ирмэг гэж нэрлэгддэг (u,v) хэлбэрийн хосын эрэмбэтэй олонлог.

(u,v) – хос хэлбэр u оройгоос v орой хүртэл ирмэг байгааг харуулж байна. Ирмэгүүд нь жин/үнэ/өртөг гэсэн мэдээллийг агуулсан байж болно.

Графыг бодит амьдрал дээр олон программд ашигладаг ба сүлжээг дүрслэхэд хэрэглэгддэг. Сүлжээнд хот доторх замууд эсвэл утасны сүлжээ түүнчлэн хэлхээний сүлжээ ч байж болно.

Доорх 2 нь графыг хамгийн түгээмэл хэрэглэдэг дүрслэл юм. Үүнд:

1. Хажуугийн матриц
2. Хажуугийн жагсаалт

Хиймэл оюун ухааны хайлтын алгоритмын төрөл

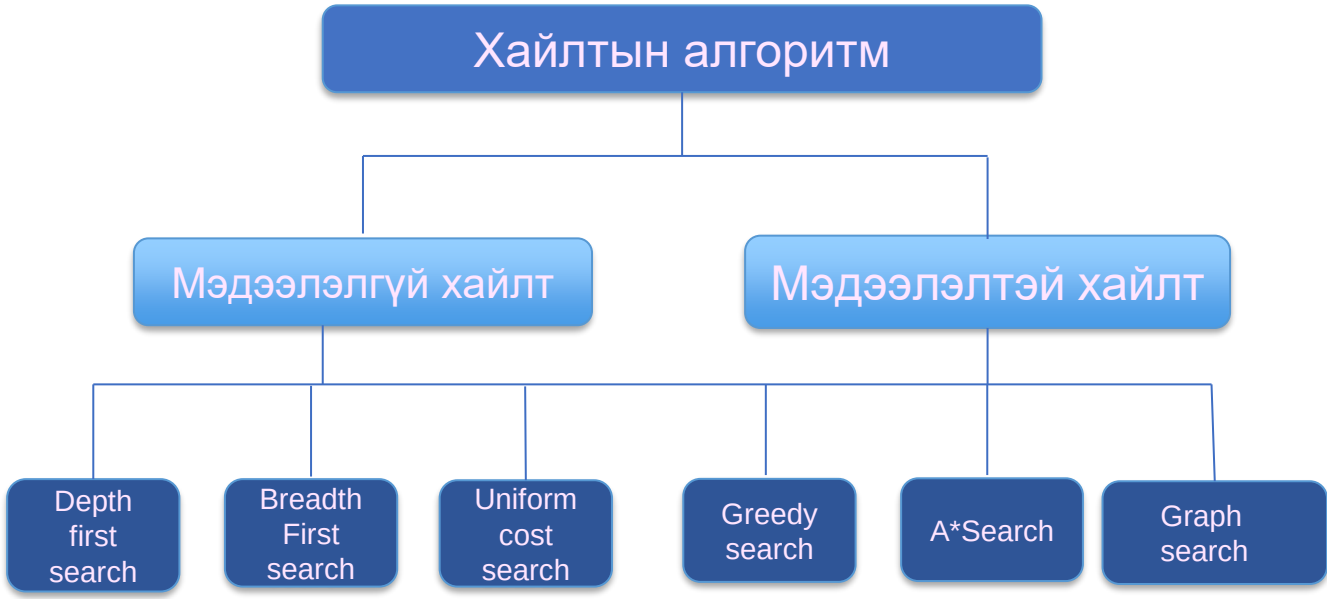
Хайлтын алгоритмууд нь эхлэлийн төлөвөөс зорилгын төлөв хүртэлх зам эсвэл шийдлийг олохтой холбоотой асуудлыг шийдвэрлэхэд зориулагдсан хиймэл оюун ухааны суурь болдог.

Эдгээр алгоритмуудыг ерөнхийд нь хоёр төрөлд хуваажболно: **мэдээлэлгүй (blind) хайлтын алгоритмууд**

болон **мэдээлэлтэй (heuristic/ эвристик) хайлтын алгоритмууд**.

Маш олон хүчирхэг хайлтын алгоритм байдаг.

Үндсэн хайлтын зургаан алгоритмыг авч үзэх болно.



Зураг 1. Хайлтын алгоритмын төрөл

Мэдээлэлгүй хайлтын алгоритмууд

Мэдээлэлгүй хайлтын алгоритмууд нь асуудлын тодорхойлолтоос өөр зорилтот төлөвийн талаар нэмэлт мэдээлэлгүй байдаг. Тэд байгаа олох хүртлээ хайлтын орон зайг сохроор судалдаг.

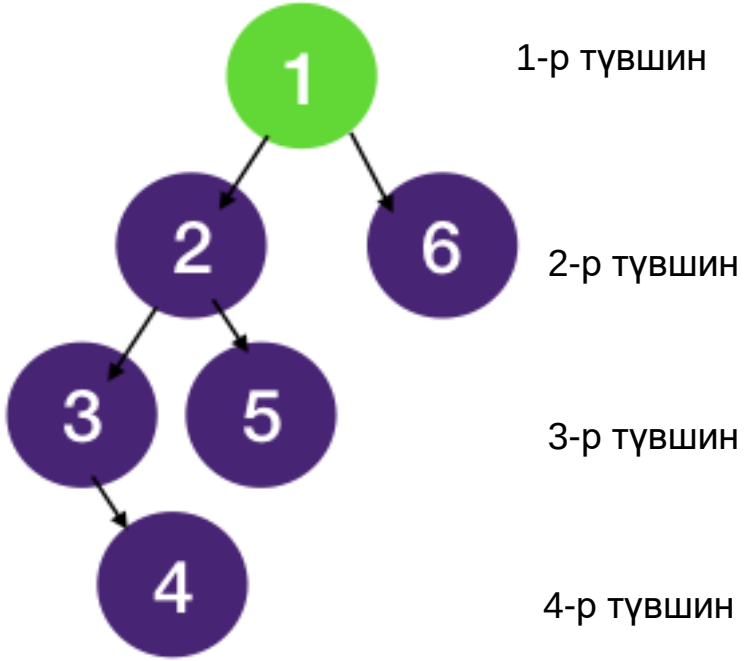
Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритмууд

Гүний хайлт –DFS

Энэ хайлт нь хайлтыг өргөтгөх, жагсаалтад сүүлд оруулсан өргөтгөх зангилааг эхэнд өргөтгөх замаар гүйцэтгэдэг буюу LIFO стекийн зарчмаар ажилладаг.

Шаардлагатай хамгийн их санах ой нь O(b*d), t=O(b^d)

DFS нь бүх зангилааг бүхэлд нь давах, боломжтой бол урагшлах, эсвэл буцах замаар графыг тойрон гүйлгээ хийдэг, ухрахад суурилсан алгоритм юм. Буцаж байна гэдэг нь бид тухайн замд зочлоогүй зангилаа үлдэх хүртэл одоогийн замаар урагшилж, дараа нь дахин зочилоогүй өөр зам руу шилжинэ гэсэн үг.



Зураг 2. DFS Гүний хайлтын ажиллагаа
Эх сурвалж:
https://miro.medium.com/v2/resize:fit:1400/format:webp/1*qA9jrd2JV0JL0FTbZrCzZw.gif

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритмууд

Мэдээлэлгүй хайлтын алгоритмууд

Гүний хайлт –DFS

DFS ашиглан шийдэж болох зарим асуудлуудыг тоочвол:

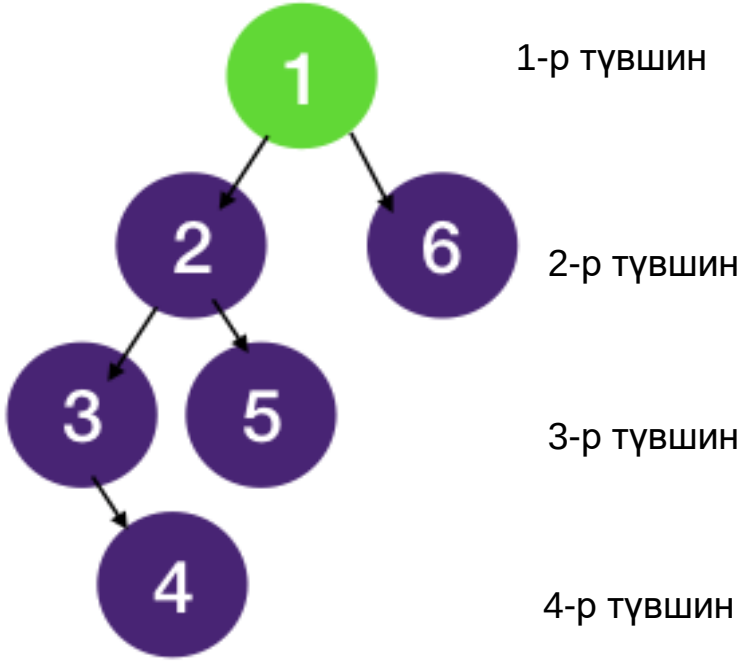
- Граф дахь холбогдсон бүрэлдэхүүн хэсгүүдийн тоог тоолох, Чиглэлгүй граф дахь давталтыг илрүүлэх, Графын топологийн эрэмбийг олох гэх мэт байж болно.

Давтамжтай Гүний хайлт –IDFS

Энэ нь хайлтын гүнийг 0 –с эхлэн C-н гүнд эхэлж хайлт хийгээд аажмаар C гүнийг нэмэгдүүлэн зорилгоо олох хүртэл хамгийн их гүнд хайх хайлт юм.

Гүнийг хязгаарласан хайлт –DLS

Энэ нь өргөний хайлтын максимум M гүнийг оптималь L гүнийг хязгаарлан хайх хайлт юм.



Зураг 2. DFS Гүний хайлтын ажиллагаа
Эх сурвалж:
https://miro.medium.com/v2/resize:fit:1400/format:webp/1*qA9jrd2JV0JL0FTbZrCzZw.gif

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм

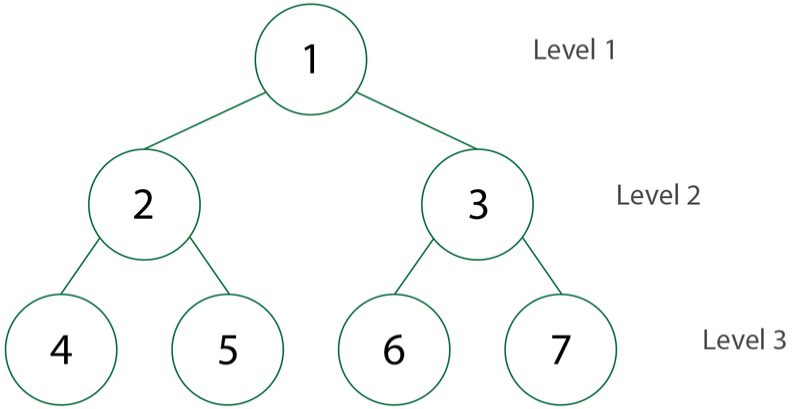
Гүний хайлт –DFS

```
def depth_first_tree_search(problem):
  #Эхлээд хайлтын модны хамгийн гүн цэгүүдийг хайж олох.
  #Зорилгоо олохын тулд асуудлын залгамж зангилааг хайж олно.
  frontier = [Node(problem.initial)]  # Stack
  while frontier: node = frontier.pop()
        if problem.goal_test(node.state):
              return node
  frontier.extend(node.expand(problem))
 return Non
```

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм

Өргөний хайлт –BFS

Энэ хайлт нь эх цэгээ өргөтгөх энгийн стратеги бөгөөд дараа нь эх зангилааны залгамжлагч,дараа нь тэрний залгамжлагч гэх мэт өргөтгөгдөөд явдаг. Энэ хайлтад модны аргаар өргөтгөхдөө жагсаалтад эхэнд орсноо түрүүлж гаргаад, сүүл талд нь шинээр өргөтгөх зангилаануудаа оруулах замаар гүйцэтгэдэг буюу (FIFO дараалал) зарчмаар явагддаг.



Зураг 3. BFS Өргөний хайлтын ажиллагаа
Эх сурвалж: https
https://miro.medium.com/v2/resize:fit:828/format:webp/1*rfMItKM-6hle3-VzrJnRCA.gif

Хиймэл оюун ухаанд Өргөний хайлт (BFS) алгоритм нь янз бүрийн асуудалтай орон зайг судлах/асуудалтай орон зайд хайгуул, навигаци хийх чухал хэрэгсэл юм. Граф эсвэл модны бүтцийг системтэйгээр дамжуулснаар BFS нь зам хайх, сүлжээний чиглүүлэлт хийх, оюун ухааны сорилт шийдвэрлэх зэрэгт ашиглагддаг.

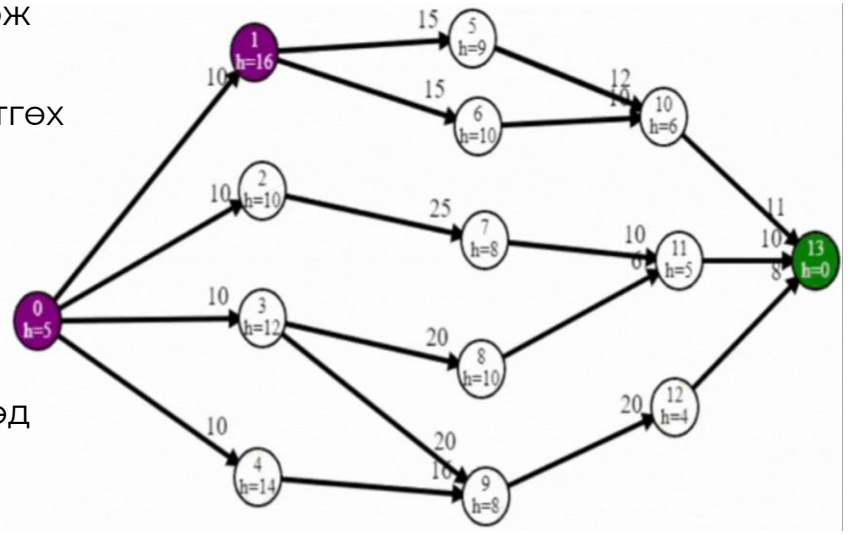
Breadth-First Search нь ХОУ болон компьютерийн шинжлэх ухааны үндсэн алгоритм бөгөөд түвшин тус бүрээр граф дамжуулалт, жингүй графаас хамгийн богино замыг олох чадвартайгаараа онцлог юм. Түүний энгийн байдал, баталгаа нь түүнийг янз бүрийн хэрэглээнд үнэ цэнэтэй хэрэгсэл болгодог. BFS-ийг ойлгох нь ХОУ дахь илүү дэвшилтэт хайлтын арга техник, граф алгоритмыг ойлгох түлхүүр юм.

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм

Өртөгөөр хайлт - Uniform cost search UCS

Энэ хайлт нь замын өртөгөөр зангилааг өртгөтгөх зарчмаар богино буюу бага өртөгтэй замыг олдог. Өөрөөр хэлбэл, g(n) = эхлэх цэгээс n зангилаа хүртэлх замын өртөг гэж авч үздэг. Dijkstra-н нэг төрлийн нэг төрлийн зардлын хайлтын алгоритмын нэг хувилбар юм. G-ийн утгыг нэмэгдүүлэх замаар өргөтгөх зангилааг эрэмбэлж өргөтгөх дараалалд оруулна. Эрэмбэлсэн дарааллаас хамгийн өндөр эрэмбэтэй зангилааг түрүүлж өргөтгөнө. Үүнд:

- Үндэс зангилааг эхлээд өргөтгөх дараалалд оруулна;
- Дараалал хоосон биш байхад давтана ;
- Хамгийн эрэмбэ өндөртэй зангилааг өргөтгөх дарааллаас гаргана ;
- Хэрэв гаргасан зангилаа зорилгын зангилаа бол нийт замын нийт зардлыг хэвлээд алгоритмыг зохионо;
- Үгүй бол уг зангилааны бүх хүү зангилаануудыг өртөгөөр эрэмбэлэн дараалалд оруулна.



Зураг 4. UCS - Өртөгөөр хайх ажиллагаа
Эх сурвалж:
https://miro.medium.com/v2/resize:fit:1400/format:webp/1*xJMKYrlbtPDMdRrFVmVm1w.gif

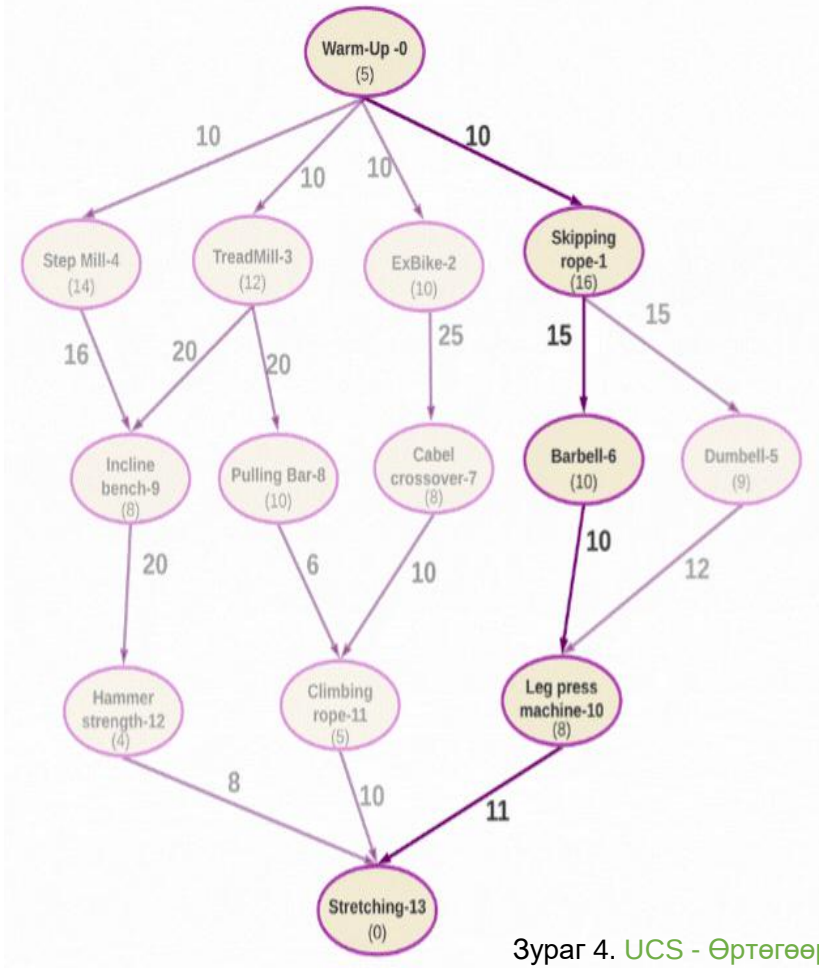
UCS-г хэрэглэх

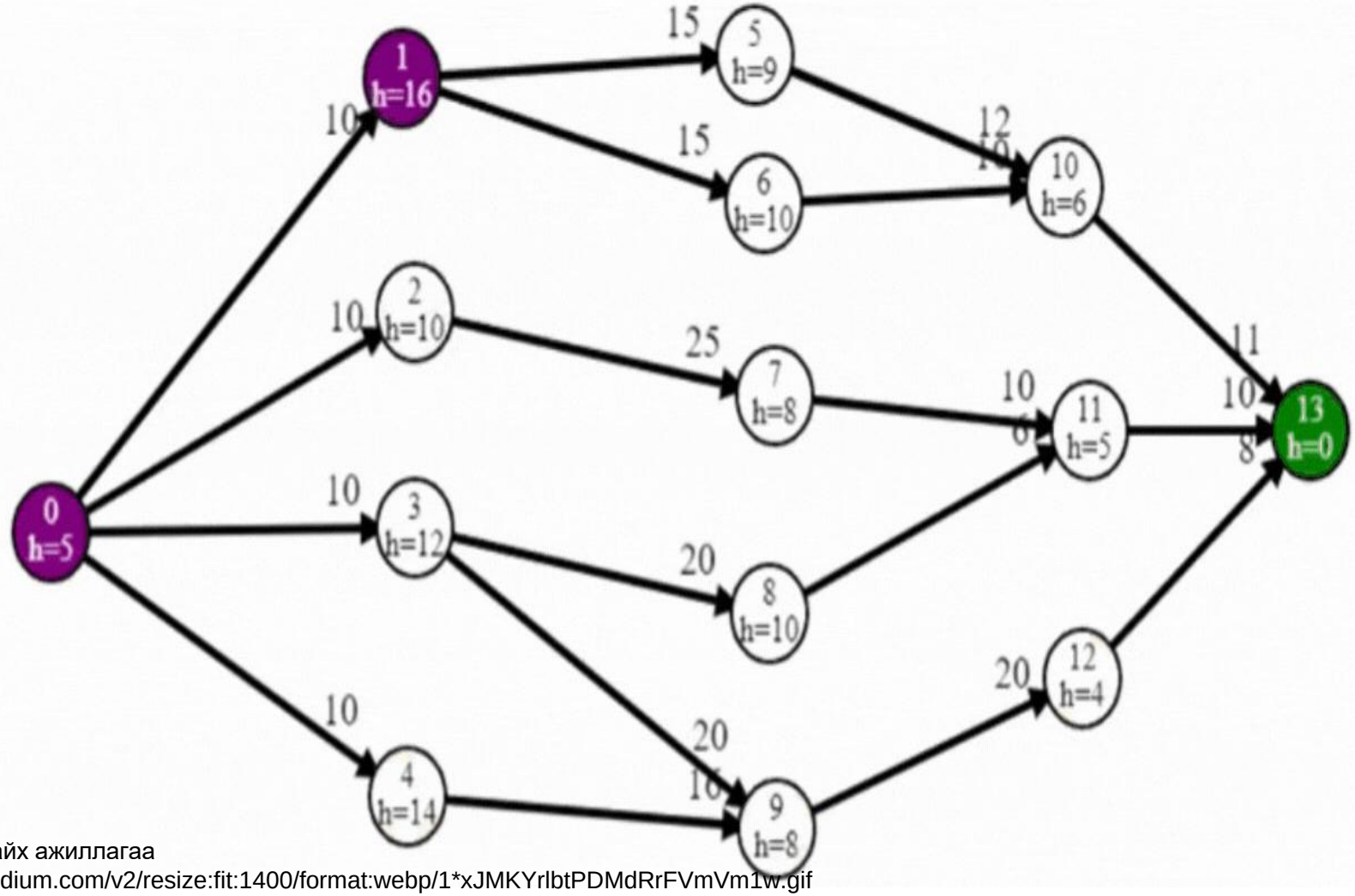
- Та хамгийн бага өртөгтэй замыг олохыг хүсвэл: UCS нь графаар дамжих хамгийн хямд замыг олоход чиглэсэн алгоритм бөгөөд ирмэгүүд нь өөр өөр зардалтай байдаг.
- Захын зардал чухал үед: Замын нийт зардал нь алхамын тооноос илүү чухал үед UCS-ийг ашиглана.
- Танд оновчтой шийдэл хэрэгтэй үед: Хэрэв таны олсон шийдэл хамгийн сайн байх баталгааг шаардаж байгаа бол UCS нь сайн сонголт юм.

.

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм

Өртөгөөр хайлт - Uniform cost search UCS





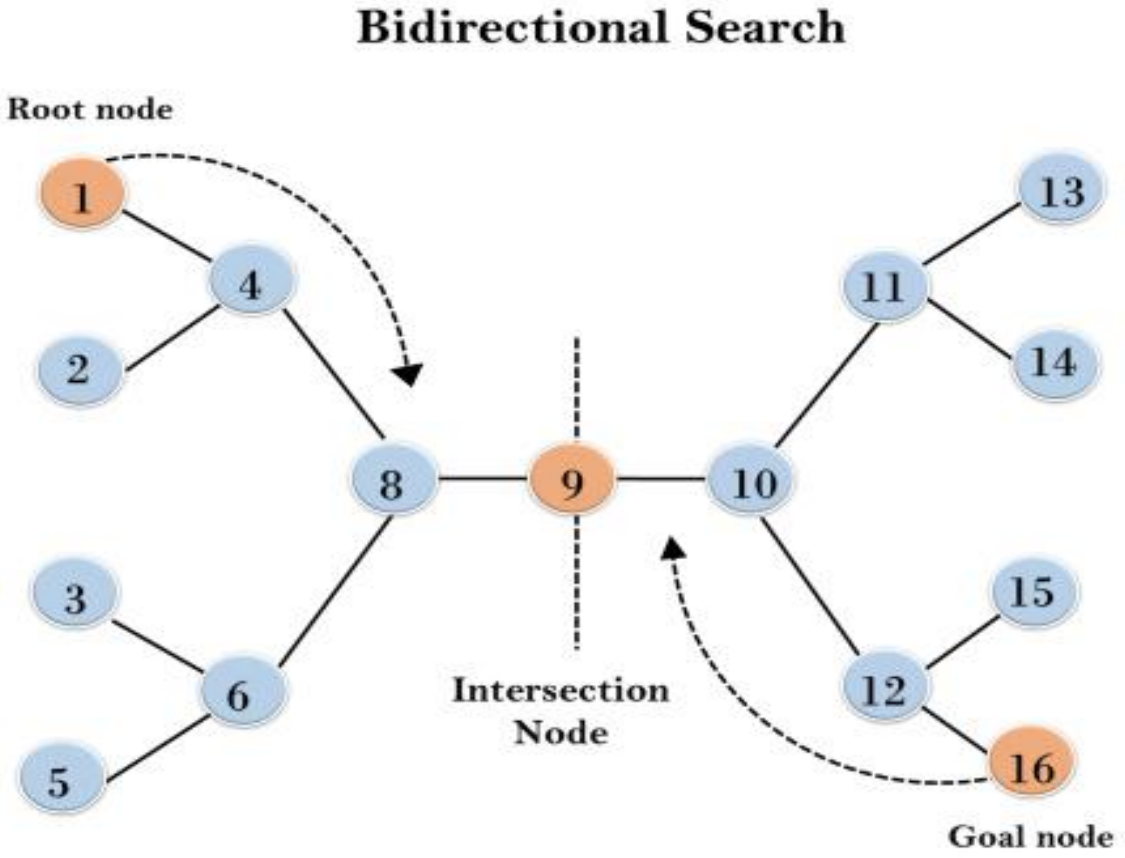
Зураг 4. UCS - Өртөгөөр хайх ажиллагаа
Эх сурвалж: https://miro.medium.com/v2/resize:fit:1400/format:webp/1*xJMKYrlbtPDMdRrFVmVm1w.gif

Хиймэл оюун ухааны мэдээлэлгүй хайлтын алгоритм

Хоёр чиглэлтэй хайлт- BDS

Хоёр чиглэлтэй хайлтын алгоритм нь хоёр хайлтыг нэгэн зэрэг явуулдаг бөгөөд зорилгын зангилааг олохын тулд нэг хэлбэр нь эхлэлийн зангилаанаас урагш хайх, нөгөө нь зорилгын зангилаанаас арагш хайх гэж нэрлэдэг. Хоёр чиглэлтэй хайлт нь нэг хайлтын графыг хоёр жижиг дэд графуудад хувааж, хайлтыг эхлэлийн оройноос, нөгөө нь зорилгын оройноос эхэлдэг. Эндгээр графууд хоорондоо огтлолцоход хайлт зогсдог.

Хоёр чиглэлтэй хайлт нь BFS, DFS, DLS стратегиудыг ашигладаг.



Зураг 5. Хоёр чиглэлтэй хайлт хийх ажиллагаа

Эх сурвалж: https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.horepnicko.cz%2Fcategory%2Faktuality%2Fpage%2F2%2F%3Fh%3Dhor6-28-4134745-5-17-44-bidirectional%2Bsearch%2Balgorithm%2Bin%2Bartificial%2Bintelligence%2Bwith%2Bexample&psig=AOvVaw0gcHzJcU3YB63_GMb6UfOY&ust=1737889247754000&source=images&cd=vfe&opi=89978449&ved=0CBQQjRxqFwoTCJiz0pTckIsDFQAAAAAdAAAAABAZ

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Мэдээлэлтэй хайлтын алгоритмуудад доорх алгоритмууд багтана.

- Greedy Search
- A* Tree Search
- A* Graph Search

Greedy Search

Greedy Search гэж нэрлэгддэг хиймэл оюун ухаанд (AI) өргөн хэрэглэгддэг хайлтын алгоритм нь үр ашигтай байхыг эрмэлздэг боловч оновчтой байдлыг золиосолдог. Энэ нь зангилаа нь зорилтот цэгт ойр байгааг харгалзан эрэл хайгуулыг эрэмбэлдэг эвристик хайлтын арга юм.

Greedy Search-үндсэн санаа бол алхам бүрт зорилтот зангилаанд хамгийн ойр харагдах зангилааг сонгох явдал юм. Энэ нь одоогийн зангилаа хүрэх зардал эсвэл зорилгодоо хүрэх зардлыг харгалзахгүйгээр энэхүү "ойрхон байдлыг" тооцохдоо эвристик функцийг ашигладаг. Алгоритм нь алхам тутамдаа дотооддоо хамгийн сайн сонголтыг анхаарч, гадаад том хүрээнд сайн шийдэлд хүргэнэ гэж найдаж байгаа учраас "шунадаг" байна.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

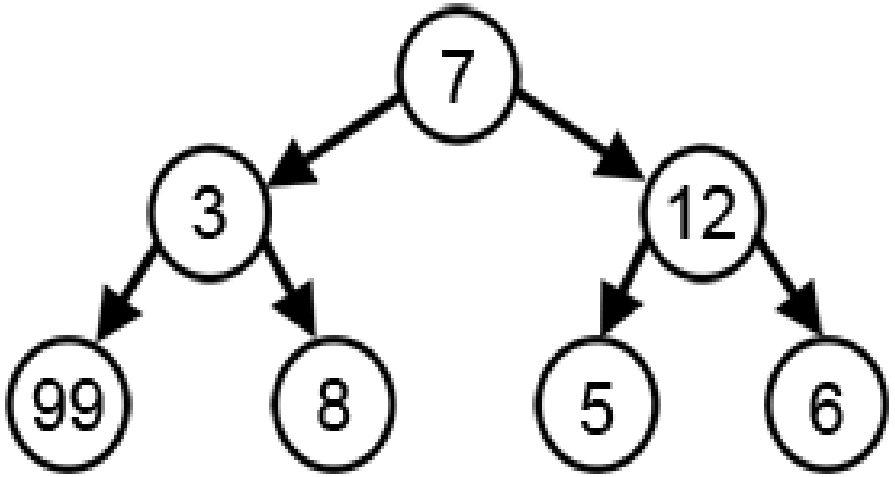
Greedy Search ажиллагаа

Эхлэл

График, эхлэлийн зангилаа, зорилтот зангилаа (та эвристикийн хувьд зорилтот зангилаа хэрэгтэй), зорилтот хүрэх зайг тооцоолох эвристик функцээс эхэлнэ.
Зангилааг эвристик утгаараа эрэмбэлэхийн тулд тэргүүлэх дараалал (эсвэл мин-овоо) үүсгэнэ.
Эхлэх зангилааг эвристик утгаараа тэргүүлэх дараалалд оруулна. Мөн давталтаас зайлсхийхийн тулд зочилсон зангилааг бүртгэнэ. Эхэндээ зөвхөн анхны зангилаа нь "зочилсон“ зангилаа байна.

Үндсэн давталт:

Тэргүүлэх дараалал хоосон биш байхад:
Dequeue: Хамгийн бага эвристик утга бүхий тэргүүлэх дарааллаас зангилаа арилгах (болон боловсруулах).



Зураг 6. Greedy Search аргаар хайлт хийх үйл ажиллагаа
Эх сурвалж: https://upload.wikimedia.org/wikipedia/commons/8/8c/Greedy-search-path-example.gif

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Greedy Search ажиллагаа

Зорилго шалгах: Хэрэв дараалалгүй зангилаа нь зорилгын зангилаа бол хайлтаа зогсоож, зам эсвэл зардлыг буцаана.
Хөршөө хайх: Зангилаагүй зангилааны зочилоогүй хөрш бүрийн хувьд: Эвристик функцийг ашиглан хөршийн эвристик утгыг тооцоол.
Хэрэв хөрш өмнө нь очиж үзээгүй бол: Хөршөө зочилсон гэж тэмдэглэнэ. Хөршийг тооцоолсон эвристик утгатай холбож, тэргүүлэх дараалалд оруулна.

Дуусгах:

Тэргүүлэх дараалал хоосон (зорилго руу хүрэх зам байхгүй гэсэн үг) эсвэл зорилгын зангилаа тэргүүлэх дарааллаас хасагдсан үед алгоритм дуусдаг бөгөөд энэ үед олдсон зам нь ихэвчлэн оновчтой биш боловч шийдэл болдог.

Greedy Search-г хэзээ ашиглах вэ

Хурд нэн тэргүүнд тавигдах үед: Хурд чухал бөгөөд танд баталгаатай оновчтой шийдэл хэрэггүй үед энэ хайлтыг ашиглана.
Сайн эвристик байгаа үед: Хэрэв та зорилтот объект хүртэлх зайг нарийн тооцоолж чадах сайн эвристик функцтэй бол хайлт үр дүнтэй байх болно.
Эвристик бодлогын хувьд: зарим нэг ойролцооллыг хүлээн зөвшөөрч болох ба оновчтой бус шийдлийг хүлээн зөвшөөрөх боломжтой асуудлуудад тохиромжтой.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Greedy Search ажиллагаа

Түлхүүр бүрэлдэхүүн хэсгүүд

Эвристик функц (h(n)): Өгөгдсөн 'n' зангилаанаас зорилгын зангилаа хүртэлх зардлыг тооцдог функц. Greedy Search-н гүйцэтгэл нь эвристик функцийн чанараас ихээхэн хамаардаг. Тооцоолоход тооцооллын хувьд хямд байх ёстой.

Priority Queue (эсвэл Min-Heap): Зангилаануудыг эвристик утгууд дээр үндэслэн хадгалдаг тул зорилтот зорилтын хамгийн бага тооцоолсон өртөгтэй зангилааг үргэлж хамгийн түрүүнд сонгоно.

Зочилсон багц (буюу жагсаалт): Давхардсан айлчлал, мөчлөгөөс зайлсхийхийн тулд өмнө нь зочилсон зангилааг бүртгэхэд ашигладаг.

A* Хайлттай холбоотой байдал

Greedy хайлт ба A* хайлт нь ижил төстэй механизмуудыг ашигладаг. A* хайлтыг мэдээлэлтэй хайлтын алгоритм гэж үздэг бол Greedy хайлтыг эвристик хайлт гэж үздэг. A* болон Greedy алгоритм хоёулаа эвристикийг ашигладаг боловч A* нь зангилааны хуримтлагдсан зардал болон зорилгын эвристикийг харгалзан үздэг бол Greedy алгоритм нь зөвхөн зорилгодоо хүрэх эвристик дээр төвлөрдөг.A* нь ихэвчлэн оновчтой шийдэлд нийлдэг бол Greedy алгоритм нь тийм биш юм.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

A* хайлтын үндсэн санаа

A* (А –од гэж дууддаг) хайлт нь өртөгөөр хайх арга болон **Greedy** хайлтын давуу талыг хослуулсан. Энэ нь хоёр бүрэлдэхүүн хэсэгтэй зардлын функцийг ашигладаг:

g(n): эхлэлийн зангилаанаас одоогийн зангилаа (n) хүртэлх замын бодит зардал.

h(n): одоогийн зангилаа (n)-аас төгсгөлийн зангилаа хүртэлх тооцоолсон зардал (эвристик функцийг ашиглан).

Дараа нь A* хайлт нь f(n) = g(n) + h(n) гэж тооцоолох замаар зангилаануудыг үнэлдэг бөгөөд энэ нь тухайн 'n зангилаагаар дамжин эхлэх цэгээс төгсгөлийн зангилаа хүртэлх хамгийн хямд замын нийт зардлын тооцоо юм. Энэ нь судлахад хамгийн бага f(n) утга бүхий зангилааг үргэлж сонгож, үр ашигтай, оновчтой алгоритм болгодог. Эхэндээ зөвхөн эхлэлийн зангилааг зочилсон гэж үзнэ.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

A* хайлтын үндсэн санаа



Зураг 7. A* хайлт хийх үйл ажиллагаа
Эх сурвалж: https://blog.finxter.com/wp-content/uploads/2021/12/Python-blog-A_star-Algorithm.gif

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

A* хайлтын үндсэн давталт

Үндсэн давталт:Тэргүүлэх дараалал хоосон биш байхад: n зангилаа f(n)-тэй тэнцүү тэргүүлэх чиглэлээр өргөтгөгдөнө. хамгийн бага
Dequeue: Алгоритмын мөчлөг бүрт хамгийн бага f(n) утга бүхий зангилааг эхлээд өргөтгөөд дараа нь дарааллаас хасагдана.
Зорилтот цэгийг шалгах: Хэрэв алсын зангилаа зорилтот цэг бол хайлтаа зогсоож, замыг сэргээнэ (шаардлагатай бол).
Хөршийн хайлт: Алсын зангилааны зочилоогүй хөрш бүрийн хувьд: Одоогийн зангилаанаас хөрш рүү чиглэсэн ирмэгийн өртгийг g (одоогийн) зардал дээр нэмж g(хөрш) зардлыг тооцоолно. Эвристик функцийг ашиглан h(хөрш)-ийг тооцоолно. f(хөрш) = g(хөрш) + h(хөрш)-ийг тооцоолно. Хөрш нь аль хэдийн очсон эсэхийг шалгана. Хэрэв тийм бол, тооцоолсон шинэ зам нь хадгалагдсан "g" тооцооноос илүү сайн эсэхийг шалгаад g оноог шинэчилж, тэргүүлэх дараалалд оруулна. Хэрвээ хөршдөө зочлоогүй бол түүнийг g, h, f оноотой тэргүүлэх дараалалд оруулна.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

A* граф хайлтыг дуусгах

Тэргүүлэх дараалал хоосон (зам байхгүй) эсвэл зорилтот зангилаа тэргүүлэх дарааллаас хасагдсан үед алгоритм дуусдаг. Зорилтот цэгийг дараалалгүй болгосны дараа бүртгэгдсэн "эцэг эх" мэдээллийг ашиглан зорилтот зангилаанаас эхлэлийн зангилаа руу буцах замаар оновчтой замыг олж авч болно.

A* хайлтыг хэзээ ашиглах вэ?

Тоглоомын зам хайх: Тоглоомын баатруудын тоглоомын ертөнцөөр аялах хамгийн дөт замыг олох.
Робот техник: Нарийн төвөгтэй орчинд роботуудын зам төлөвлөлт.
Маршрут төлөвлөлт: Навигацын систем дэх хамгийн дөт замыг олох (жишээ нь, Google Maps).
Логистик: Ачааны машин эсвэл дроны хүргэх замыг оновчтой болгох.
Сүлжээний чиглүүлэлт: Сүлжээгээр дамжих өгөгдлийн пакетуудын оновчтой замыг олох. Оновчтой шийдлийг үр дүнтэй олох шаардлагатай график хайлтын аливаа асуудал.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Бусад хайлтын алгоритмуудтай харилцах

BFS (Breadth First Search): A* нь эвристикийг ашигладаг тул BFS-ээс илүү ухаалаг байдаг. BFS нь ирмэгийн тооны хувьд оновчтой замыг баталгаажуулдаг боловч A* нь зардлын хувьд оновчтой замыг баталгаажуулдаг.

UCS (Өртөгөөр хайх хайлт): Эвристик функцийг тэг болгож тохируулсан үед A*-г UCS-ийн өргөтгөл гэж үзэж болно. Бүх зангилааны хувьд эвристик 0 байх үед A* нь UCS-тэй адилхан ажиллах бөгөөд бүрэн бөгөөд оновчтой байна.

Greedy хайлт: A* нь бодит зардал (g-оноо) болон эвристик зардлыг хоёуланг нь авч үздэгээрээ Greedy хайлтаас ялгаатай, харин Greedy хайлт нь зөвхөн эвристик зардлыг тооцдог. A * ихэвчлэн илүү оновчтой (мөн илүү бүрэн гүйцэд).

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Graph хайлт

Хиймэл оюун дахь график хайлт нь үндсэндээ тодорхой шалгуурыг хангасан зам, шийдэл эсвэл холбогдох мэдээллийг олохын тулд график өгөгдлийн бүтцийг судлах алгоритмыг ашиглахыг хэлнэ. Асуудлыг шийдвэрлэхийн тулд харилцан уялдаатай байгууллагуудын сүлжээг стратегийн хувьд даван туулах тухай юм.

Графикууд нь дүрслэл юм:

Зангилаа (орой): асуудалд байгаа объект, объект эсвэл төлөвийг төлөөлдөг (жишээ нь, газрын зураг дээрх байршил, тоглоомын төлөв, өгүүлбэр дэх үгс).

Ирмэгүүд (холболт): зангилаа хоорондын харилцаа, шилжилтийг илэрхийлдэг (жишээ нь, байршлын хоорондох зам, албан ёсны тоглоомын хөдөлгөөн, үгсийн хоорондын утгын холбоо).

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

График хайлтын ач холбогдол

График хайлт нь дараах тохиолдолд асуудлыг загварчлах, шийдвэрлэх үр дүнтэй арга учраас Хиймэл оюун ухааны үндсэн суурь юм.

Харилцаа чухал: Бодит ертөнцийн олон асуудал нь бүрэлдэхүүн хэсгүүдийн хооронд нарийн төвөгтэй харилцаа, холбоотой байдаг.

Нөхцөл байдал ба шилжилтийг тодорхойлсон: Хиймэл оюун ухааны олон асуудлыг нарийн тодорхойлсон үйлдэл эсвэл шилжилтийг ашиглан эхний төлөвөөс зорилгын төлөв рүү хайх хэлбэрээр томъёолж болно.

Хамгийн сайн зам эсвэл дарааллыг олох: Олон асуудал нь оновчтой шийдлийг олохыг шаарддаг бөгөөд ихэнхдээ зам эсвэл шийдлийн цуврал байдаг.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

График хайлтын асуудлын гол бүрэлдэхүүн хэсгүүд

Төлөвийн орон зай: даалгавар байж болох төлөв эсвэл тохиргооны бүхэл бүтэн багц.

Анхны төлөв: хайлтын эхлэлийн цэг.

Зорилтот төлөв(үүд): Хүссэн эцсийн төлөв(үүд) эсвэл таны хүрэхийг хүсэж буй нөхцөл.

Үйлдлүүд эсвэл операторууд: Нэг төлөвөөс нөгөөд шилжих боломжит шилжилтүүд.

Зардлын функц (заавал биш): Үйлдэл хийх эсвэл төлөв хоорондын шилжилтийн зардал (жижиглэсэн графикийн асуудалд ихэвчлэн ашиглагддаг).

Эвристик функц (заавал биш): Зорилго хүртэлх зайг тооцоолох арга (мэдээлэлтэй хайлтанд ашигладаг).

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

График хайлтын алгоритмын төрлүүд

Uninformed (Blind) Search : Эдгээр алгоритмууд нь асуудлын тодорхойлолтод тусгагдсанаас бусад тохиолдолд зорилгын төлөвийн талаарх тодорхой мэдээллийг ашигладаггүй.

Breadth-First Search (BFS): Графикийн түвшинг түвшингээр нь судалдаг; хамгийн богино замыг олоход жингүй графикт оновчтой.

Depth-First Search (DFS): Салбар бүрийн дагуу аль болох гүнзгий судалдаг; санах ойн хэмнэлттэй байж болох ч үргэлж оновчтой байдаггүй.

Uniform Cost Search(UCS): Одоогоор судлагдсан хамгийн бага зардлаар судалдаг; жигнэсэн графикийн хувьд оновчтой.

Informed (Heuristic) Search/Мэдээлэлтэй (Эвристик) хайлт: Эдгээр алгоритмууд нь хайлтаа удирдан чиглүүлэхийн тулд эвристик мэдээллийг ашигладаг.

Greedy Search (Greedy Best-First): Эвристик дээр тулгуурлан зорилгодоо хамгийн ойр байгаа зангилаануудыг өргөжүүлнэ; ихэвчлэн хурдан, гэхдээ оновчтой биш, бүрэн гүйцэд биш.

A* Хайлт: Зангилаанд хүрэх зардал болон зангилаанаас зорилго хүртэлх тооцоолсон зардлыг нэгтгэдэг ($f = g + h$); Зөвшөөрөгдөх эвристикийг ашиглах үед бүрэн бөгөөд оновчтой.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

График хайлт ашигладаг бодит программууд

Тоглоом дахь зам хайх: Тоглоомын баатруудыг газрын зураг эсвэл түвшингээр дамжуулан чиглүүлэх; A* нь үр ашиг, оновчтой замыг олох чадвараараа байнга ашиглагддаг.

Робот техник: Орчин тойрноо жолоодох, объектыг удирдах, даалгавруудыг гүйцэтгэх роботуудад зориулсан замыг төлөвлөх; Хайлтын алгоритмууд нь зам, хөдөлгөөнийг төлөвлөхөд зайлшгүй шаардлагатай.

Навигацийн систем (GPS): Газрын зураг дээрх хоёр байршлын хоорондох хамгийн дөт замыг олох; A*, UCS болон бусад алгоритмуудын навигацийн хэрэгслүүд.

Нийгмийн сүлжээний шинжилгээ: Харилцаа холбоог судлах, олон нийтийг илрүүлэх, нийгмийн сүлжээнд нөлөөлөгчдийг тодорхойлох; График хайлтын янз бүрийн алгоритмуудыг жишээ нь харилцааг тодорхойлоход ашигладаг.

Төлөвлөлт: Нарийн төвөгтэй зорилгод хүрэхийн тулд үйл ажиллагааны дарааллыг боловсруулах; График хайлтыг даалгаврын менежментийн алгоритмыг төлөвлөх хэрэгсэл болгон ашигладаг.

Хуваарь гаргах: Хязгаарлагдмал байдлыг хангахын зэрэгцээ даалгаварт нөөц хуваарилах; хайлтын алгоритмууд нь боломжтой эсвэл оновчтой хуваарийг олоход тусалдаг.

Сүлжээний чиглүүлэлт: Компьютерийн сүлжээн дэх өгөгдлийн багцын хамгийн үр дүнтэй замыг олох; Дижикстрагийн алгоритм (UCS-тэй маш төстэй) зэрэг алгоритмуудыг ашигладаг.

Puzzle Solving : Puzzle автоматаар эвлүүлэх; BFS хайлт нь үүнийг хийх боломжтой хиймэл оюун ухааныг бий болгох үндсэн арга юм.

Вэб скан хийх: Гүйлгээний холбоосыг дагаж дэлхийн сүлжээний бүтцийг судлах; BFS-ийг ихэвчлэн ашигладаг.

Хиймэл оюун ухааны мэдээлэлтэй хайлтын алгоритм

Хиймэл оюун ухаан дахь график хайлтын үүрэг

Асуудлыг шийдвэрлэх: График хайлт нь хиймэл оюун ухааны олон төрлийн асуудлыг төлөөлөх, шийдвэрлэх үндсэн хүрээг бүрдүүлдэг.
Шийдвэр гаргах: Хайлтын алгоритмууд нь хиймэл оюун ухааны агентуудад боломжит сонголтууд болон тэдгээрийн үр дагаврыг судлах замаар оновчтой эсвэл ойролцоо оновчтой шийдвэр гаргахад тусалдаг.
Үндэслэл: График хайлтыг мэдлэгийн дүрслэл дэх холбоо, хамаарлын талаар үндэслэл гаргахад ашиглаж болно.
Төлөвлөлт ба хяналт: График хайлт нь үйлдлийн дарааллыг төлөвлөж, робот эсвэл бие даасан агентуудыг удирддагхиймэл оюун ухааны системүүдийн гол бүрэлдэхүүн хэсэг юм.

График хайлтын алгоритмыг сонгохдоо анхаарах зүйлс:

Графикийн бүтэц: Таны график жинлэсэн, жингүй, чиглүүлсэн эсвэл чиглүүлээгүй юу?
Хайлтын зорилго: Та ямар нэгэн зам, хамгийн дөт зам эсвэл хамгийн бага зардалтай замыг хайж байна уу?
Гүйцэтгэлийн шаардлага: Та хурд, оновчтой байдал эсвэл санах ойн бага хэрэглээг чухалчилдаг уу?
Эвристикийн боломж: Та хайлтыг үр дүнтэй удирдаж чадах сайн эвристик функцийг тодорхойлж чадах уу?
Графикийн хэмжээ: Шаардлагатай бол таны алгоритм том, нарийн төвөгтэй графиктай ажиллах боломжтой юу?

Дүгнэлт

Энэ удаагийн лекцээр талаарх ойлголтуудыг өгсөн болно. Үүнд:

- хайлт хийх үйлдлийн тодорхойлолт, түүнийг хийх шалтгаан, хайлт хийх аргачлал;
- Хиймэл оюунд түгээмэл хэрэглэгддэг хайлтын алгоритмууд;
- Хайлтын алгоритмыг дүрслэх хэрэгсэл;
- Хайлтын алгоритмд хэрэглэгддэг хэллэгүүд, Алгоритмуудын төрөл, түүнд харгалзах алгоритмуудын онцлог, ялгаа, ажиллах зарчим, тэдгээрийг хиймэл оюуны ямар төрлийн асуудлуудыг шийдвэрлэхэд хэрхэн ашиглах зэрэг ойлголтууд ;

Эдгээр ойлголтуудыг лабораторийн хичээлээр туршиж үзэн бие дааж бодлого бодох болно.

Анхаарал тавьсанд баярлалаа.

Асуултаа сургалт удирдах системийн зурвасаар илгээнэ үү.